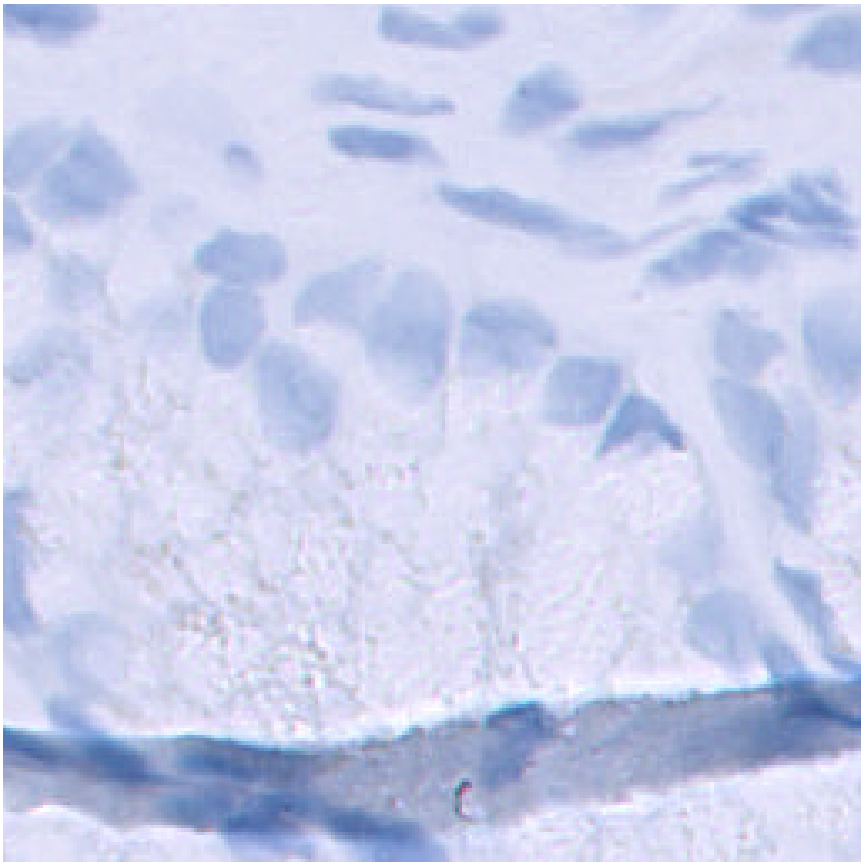
Deep Learning for the Detection of *Helicobacter pylori* in Immunohistochemical Histopathological Images

Project Report



Representative histology: IHC-stained gastric biopsy patch.

Théotime Donnenfeld	NIU 1786565
Rubén Alba	NIU 1786494
Rohit Ranbawale	NIU 1744008
Michael Rienessl	NIU 1699828

Universitat Autònoma de Barcelona (UAB)

Course: Deep Learning

March 3, 2026

Abstract

We use the Quiron Salut dataset of 245 gastric biopsies (immunohistochemically stained whole-slide images) to implement and compare two patch-level models for *Helicobacter pylori* detection. A patch-based pipeline processes 256×256 patches extracted along tissue borders. Two approaches are implemented: (i) a supervised convolutional neural network (CNN) trained on expert-annotated patches, and (ii) an unsupervised autoencoder (AE) trained on healthy patches and used for anomaly detection via reconstruction error. This report shows that both models are running and have been compared on the same validation set: we evaluate them with the comparison script (`compare_patch_classifiers.py`), report patch-level accuracy and precision, and include a side-by-side confusion-matrix plot. **Scope of this first report is patch-level comparison; patient-level aggregation and evaluation on the independent hold-out set are planned as next steps.**

1 Introduction

Helicobacter pylori is a gastric bacterium associated with chronic gastritis and an increased risk of gastric cancer. In clinical practice, detection often relies on the visual inspection of immunohistochemically (IHC) stained histopathology slides. Whole-slide images (WSI) can exceed 100 000 pixels per side, making manual review time-consuming. This project investigates automated, patch-based detection: we classify small image patches as healthy or containing bacteria, then aggregate patch predictions to obtain a patient-level diagnosis.

The pipeline has three stages: (1) train a patch-level classifier on expert-annotated data; (2) apply it to all patches of each patient and compute the fraction of positive patches; (3) compare that fraction to a threshold to yield a binary patient diagnosis. Key challenges are the limited number of annotated patches, strong class imbalance (few patches contain bacteria), and the need to avoid data leakage by splitting at the patient level rather than the patch level.

We implement two patch-level models—a supervised CNN and an unsupervised autoencoder used for anomaly detection—and compare them on the same validation set. This report describes the methodology, experimental setup, and patch-level results; patient-level aggregation and evaluation on an independent hold-out set are left for future work.

2 Dataset Description

We use the Quiron Salut dataset: IHC-stained gastric biopsy whole-slide images from which 256×256 pixel patches have been cropped along the tissue border. Patient-level labels indicate H. *pylori* density: NEGATIVE, LOW, or HIGH. For binary classification we map NEGATIVE to 0 and (LOW, HIGH) to 1.

Patch-level annotations are provided in `CoordAnnotatedAllPatches.csv` with a `Presence` field: -1 (healthy), 1 (bacteria present), or 0 (unclear, which we excluded from training as there are 3 or 4 cases only).

The data are organised into three parts: **CrossValidation/Annotated** (expert-labelled patches for training and validating patch-level models), **CrossValidation/Cropped** (all patches per patient for aggregation experiments), and **HoldOut** (116 independent patients reserved for final evaluation only). Table 1 summarises the sample counts.

Split	#Patients	#Annotated patches	#Positive patches	#Negative patches
CrossValidation	123	1211	161	1050
HoldOut	116	–	–	–

Table 1: Dataset size overview (Quiron Salut).

3 Methodology

3.1 Pipeline overview

The pipeline has three steps. First, a patch-level model (CNN or autoencoder) is trained on annotated or healthy-only patches. Second, the model is applied to every patch of each patient. Third, the fraction of patches predicted positive per patient is computed and compared to a threshold $\tau_{patient}$ to obtain a binary patient-level diagnosis. Within this first report we address patch-level classification by comparing an autoencoder and a CNN classifier. Figure 1 summarises the workflow.

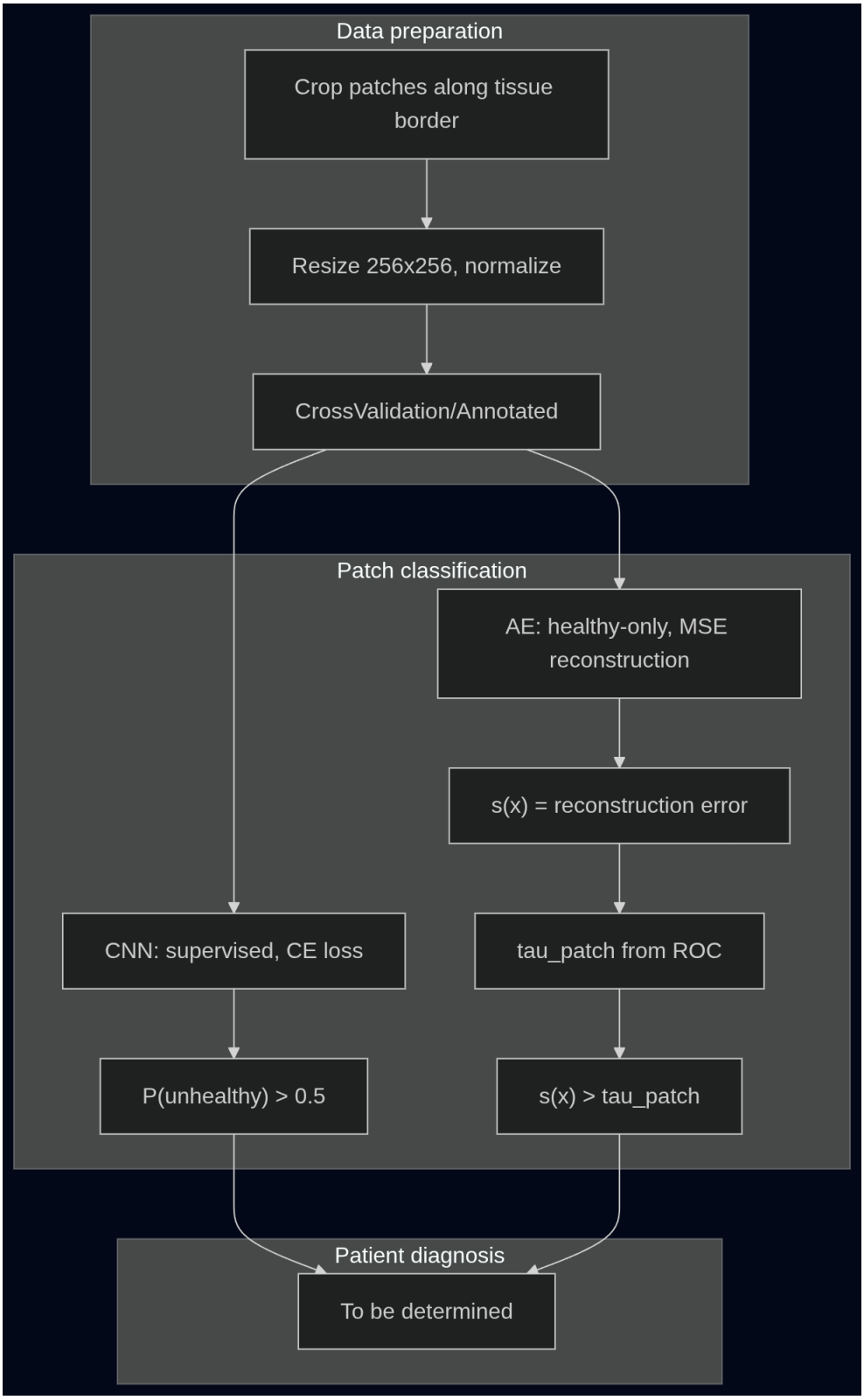


Figure 1: Methodology workflow: data preparation, patch-level classification (CNN vs autoencoder), and patient-level aggregation (to be evaluated in a later phase).

3.2 Preprocessing

Patches are loaded as RGB images, resized to 256×256 if needed, and normalised per channel with $x' = (x - \mu)/\sigma$ where $\mu = \sigma = (0.5, 0.5, 0.5)$, so that inputs lie approximately in $[-1, 1]$. For the CNN we apply data augmentation: random flips, small rotations, and optional colour jitter.

3.3 Patch-level approach 1: CNN classifier

We use a lightweight CNN with three convolutional blocks followed by fully connected layers. Training minimises cross-entropy loss. With one-hot label y and predicted class probabilities $\hat{p}_c$ (softmax over logits), the loss is

$$\mathcal{L}_{\text{CE}} = -\sum_{c \in \{0,1\}} y_c \log \hat{p}_c,$$

with $c = 0$ for healthy and $c = 1$ for patches containing H. *pylori*. The model is trained with Adam; early stopping is optional. At inference, a patch is predicted positive if the probability of class 1 exceeds 0.5.

3.4 Patch-level approach 2: Autoencoder anomaly detection

The autoencoder is trained only on healthy patches and learns to reconstruct normal tissue. Training minimises mean squared error between input and reconstruction:

$$\mathcal{L}_{\text{AE}} = \frac{1}{CHW} \sum_{c,i,j} (x_{c,i,j} - \hat{x}_{c,i,j})^2,$$

where $C = 3$, $H = W = 256$. The per-patch anomaly score $s(x)$ is the reconstruction error. We use either the mean over all pixels or the maximum over 8×8 spatial cell means (`max_local`) so that small anomalous regions contribute more. A patch is predicted positive if $s(x) > \tau_{patch}$. The threshold τ_{patch} is chosen on the validation set as the point on the ROC curve closest to (FPR= 0, TPR= 1), implemented as `roc_threshold_optimal`.

3.5 Patient-level aggregation

For each patient p with N_p patches and binary patch predictions $\hat{z}_i \in \{0, 1\}$, we define the contamination ratio

$$r_p = \frac{1}{N_p} \sum_{i=1}^{N_p} \hat{z}_i.$$

The patient is predicted positive if $r_p > \tau_{patient}$. In our implementation, $\hat{z}_i$ is derived from a patch-level score and threshold:

- CNN: score = $P(\text{unhealthy})$, threshold $\tau^{cnn}_{patch} = 0.5$ (default).
- AE: score = reconstruction error (`mean` or `max_local`), threshold τ^{ae}_{patch} from ROC (`autoencoder_threshol`
  or fold-specific threshold files).

The patient threshold $\tau_{patient}$ is selected on training patients by maximizing patient-level F1 over candidate ratio thresholds, then evaluated on validation patients. The same protocol is applied fold-by-fold in patient-level CV, and then on Hold-out.

4 Implementation Details

4.1 Development iterations

The CNN started as a minimal classifier: three small convolutional blocks (3→16→32→64 channels, ReLU, max pooling) and two fully connected layers mapping the flattened 64×32×32 features to two class logits. No architectural changes were required for good performance; the main refinements were patient-level train/validation splitting to avoid leakage, ignoring patches with `Presence=0` in the annotation file, and training for more epochs (e.g. 150 in the Slurm script). The current design remains lightweight and fast to train.

The autoencoder began with a shallower encoder (32→64→128→256→256 channels, 8×8 bottleneck) and ReLU activations. To avoid losing information in the bottleneck (ReLU zeros negatives), ReLU was replaced by LeakyReLU. The encoder and decoder were then deepened and widened (64→128→256→512→512 channels, 8×8×512 bottleneck) to improve reconstruction of healthy tissue so that anomalous patches yield higher error. The anomaly score was extended with a `max_local` option (maximum over 8×8 spatial cell means) so that focal H. *pylori* regions drive the score. Checkpoint selection was fixed to keep the best model by validation loss instead of the last epoch, and the default aggregation and epoch count were set to `max_local` and 35 (overridable via Slurm).

4.2 Code structure

The codebase is organised as follows: `model_classifier.py` defines and trains the CNN; `model_autoencoder.py` defines and trains the autoencoder (including ROC-based threshold selection); `src/helico/cv_utils.py` contains shared patient-level split and metric helpers used by both models; `compare_patch_classifiers.py` evaluates both models on the same validation split and saves confusion-matrix and ROC figures.

Patient-level diagnosis is implemented in `patient_level_pipeline.py`. This script supports two modes: (i) single-checkpoint mode, and (ii) fold-consistent mode (`--use-cv-fold-checkpoints`) that loads fold-specific models from `cv/cnn/fold_i/` and `cv/autoencoder/fold_i/` (including AE fold thresholds). Outputs are written under `output/patient_level/<model>/`: fold metrics, Hold-out-per-fold metrics, and summary statistics.

Training on a GPU cluster is launched via `train_classifier.sbatch.sh` and `train_autoencoder.sbatch.s` (legacy `train.sbatch.sh` is kept as a compatibility script). Patient-level jobs are launched via `patient_cnn_fold.sbatch.sh`, `patient_autoencoder_fold.sbatch.sh`, and `patient_plots.sbatch.sh`. Dataset paths follow the `HelicoDataSet/` layout (CrossValidation/Annotated, CrossValidation/Cropped, HoldOut). Diagnostics scripts (`analyze_cv_metrics.py`, `analyze_ae_fold_diagnostics.py`, `analyze_patient` produce plots and robust-statistics summaries.

5 Experimental Design

We split data at the **patient level** so that a patient never appears in both train and validation folds. The current evaluation uses **5-fold patient-level cross-validation** (`--k-folds 5`, `seed=42`) for both CNN and AE. For each fold, checkpoints and metrics are stored under `cv/cnn/fold_i/` and `cv/autoencoder/fold_i/`. The Hold-out set (116 patients) remains reserved for a single final evaluation after all model and threshold design choices are fixed.

For patient-level diagnosis, we compute contamination ratios on `CrossValidation/Cropped` and select fold-wise $\tau_{patient}$ from training patients only (best F1). We then evaluate on validation patients and additionally report Hold-out metrics per fold using the same fold model and $\tau_{patient}$. This avoids mixing fold artifacts and keeps patch-level and patient-level protocols consistent.

We report fold-wise and aggregate patch-level metrics (accuracy, precision, recall, specificity, F1), using both mean±std and robust summaries (median/IQR). For AE we additionally report threshold behavior, TPR and FPR by fold.

5.1 Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs (CNN)	150
Epochs (AE)	150
Weight decay	0 (not used)
Early stopping	not used
CV protocol	patient-level 5-fold, `seed=42`
Patch threshold τ_{patch}	ROC: $\arg\min_\tau \big(\mathrm{FPR}(\tau)^2 + (1-\mathrm{TPR}(\tau))^2\big)$ (`roc_threshold_optimal`)
Patient threshold $\tau_{patient}$	from CV; hold-out evaluation TBD

Table 2: Hyperparameters and training configuration (from `model_classifier.py`, `model_autoencoder.py`, and Slurm scripts).

6 Results

Both models were evaluated with patient-level 5-fold CV. Aggregate results are reported in Table 3. Figure 2 shows mean±std comparison across folds. For AE, fold-level diagnostics revealed one anomalous fold (fold 3) with an error-scale shift and much higher optimal threshold; this is detailed in Figure 3.

6.1 Patch-level metrics

Table 3 summarises patch-level performance. Recall is TP/(TP+FN) and specificity is TN/(TN+FP). The CNN attains higher accuracy and precision; the autoencoder is slightly lower but still strong, and was trained without any positive patch labels.

Method	Accuracy	Precision	Recall	Specificity	F1
CNN ($\mu \pm \sigma$)	0.9108±0.0484	0.9706±0.0197	0.8783±0.0717	0.9532±0.0499	0.9201±0.0382
Autoencoder ($\mu \pm \sigma$)	0.8246±0.0736	0.8836±0.0619	0.8065±0.1009	0.8563±0.0380	0.8397±0.0696

Table 3: Patch-level cross-validation performance (5-fold patient-level CV).

6.2 Cross-validation variability and AE fold anomaly

Figure 2 compares fold-aggregated performance between models. CNN remains stronger and more stable across folds. AE is competitive but more sensitive to fold composition and reconstruction-error scale shifts.

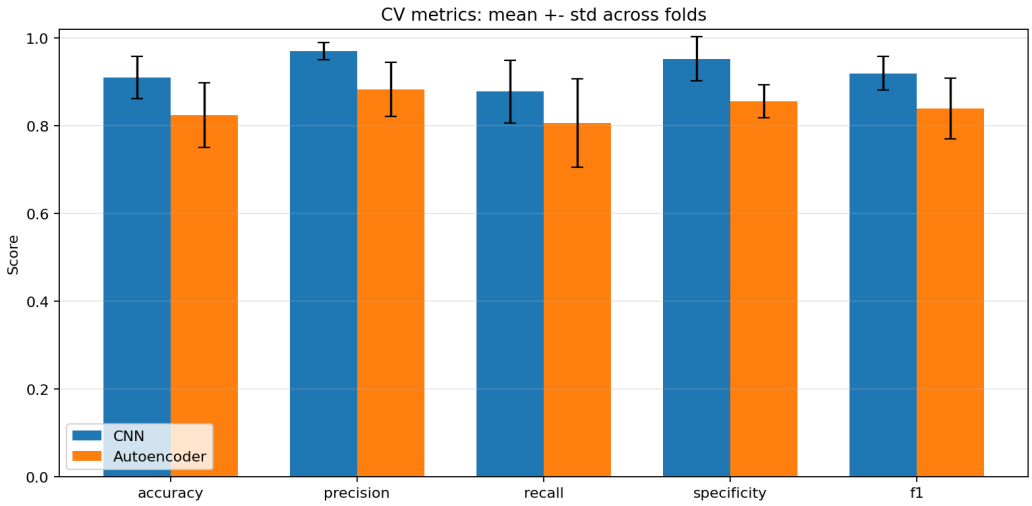


Figure 2: Model comparison across patient-level CV folds (mean±std).

AE fold diagnostics (Figure 3) show fold 3 has a distinct error scale. The fold-3 local threshold is 0.7871 while other folds are around 0.04–0.07. Class-wise reconstruction-error medians in fold 3 are also much larger (healthy median ≈0.60, unhealthy median ≈1.05), indicating a distribution shift rather than a simple class-imbalance artifact.

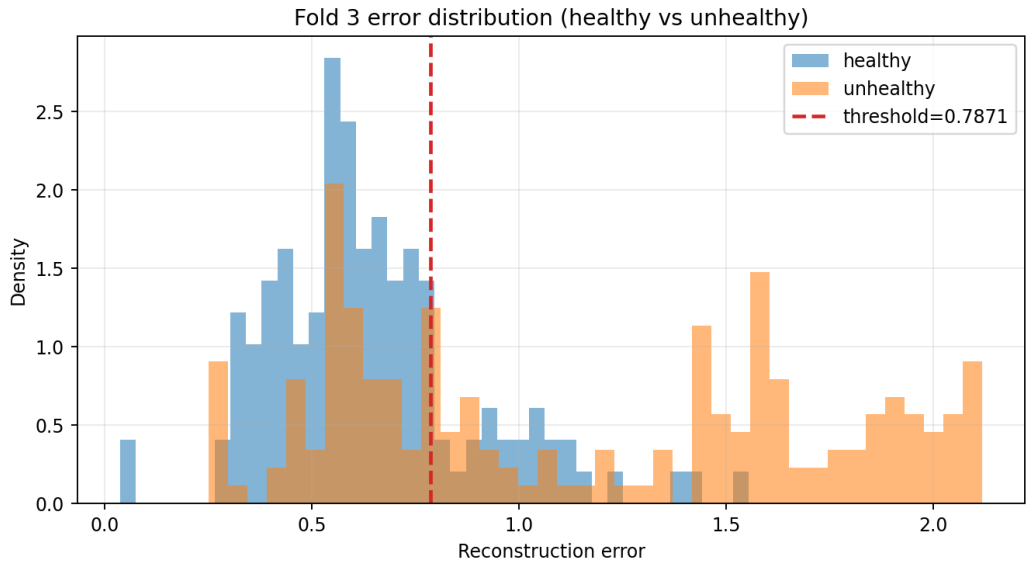


Figure 3: Fold-3 AE reconstruction-error distributions (healthy vs unhealthy) and selected threshold.

Robust summaries confirm that mean/std is distorted by outliers. For AE threshold, mean±std is 0.2092±0.2892 but the robust estimate is median 0.0717 with IQR 0.0027, showing that one outlier fold dominates the standard deviation.

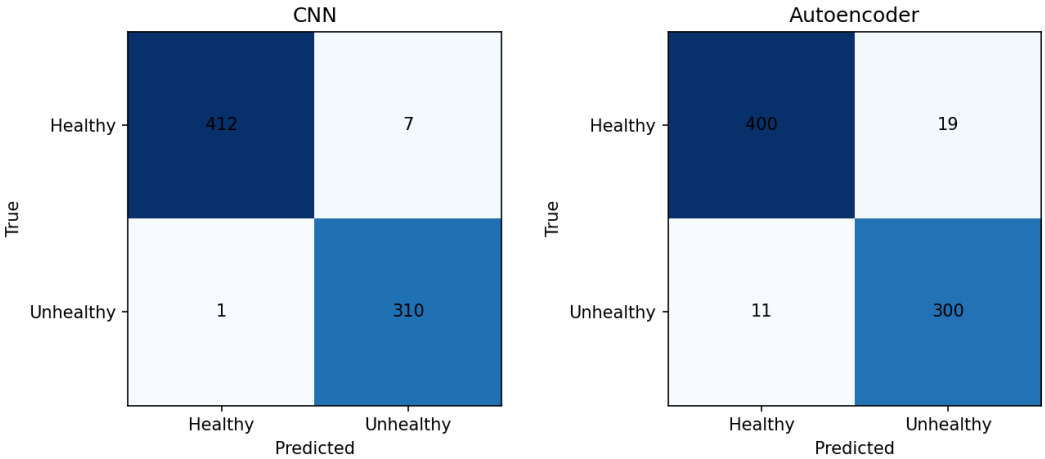


Figure 4: Confusion matrices on the validation set. Left: CNN. Right: Autoencoder (threshold from validation ROC).

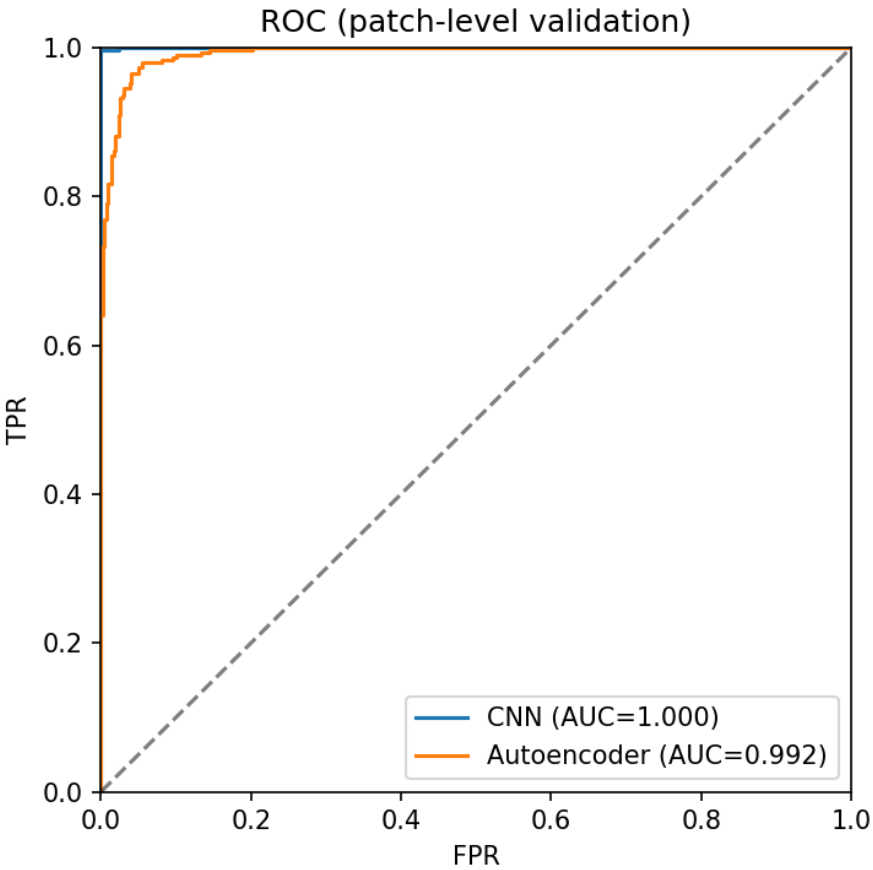


Figure 5: ROC curves on the validation set.

6.3 Patient-level

Patient-level performance is computed by `patient_level_pipeline.py`. The pipeline stores fold CV metrics and Hold-out-per-fold metrics in JSON files under `output/patient_level/`. Plots and text summaries are generated by `analyze_patient_level_metrics.py`. Table 4 is the target format for final values after cluster execution completes.

Method	Accuracy	Precision	Recall	Specificity
CNN + aggregation	–	–	–	–
AE + aggregation	–	–	–	–

Table 4: Patient-level performance (to be filled after aggregation and hold-out evaluation).

7 Discussion

With patient-level 5-fold CV, the CNN consistently outperforms the AE in accuracy/precision/F1 and exhibits lower fold-to-fold variance. The AE remains useful in low-label settings but is notably more sensitive to fold-specific data distribution.

The main observed limitation is threshold instability under distribution shift: fold 3 presents an upward shift of reconstruction errors for both classes, which inflates local threshold and degrades recall/F1. A pooled global threshold improved some folds but harmed specificity severely on fold 3, so a single global threshold is not yet robust enough. Next steps are threshold normalization/calibration strategies (e.g., robust scaling), patient-level aggregation, and final Hold-out evaluation.

8 Conclusion

We implemented two patch-level models for H. *pylori* detection: a supervised CNN and an unsupervised autoencoder for anomaly detection, and evaluated both with patient-level 5-fold CV.

The CNN achieves stronger and more stable performance (F1 0.9201 ± 0.0382) than the AE (F1 0.8397 ± 0.0696). AE diagnostics identify a fold-specific error-scale anomaly (fold 3), motivating robust threshold calibration rather than a single pooled threshold.

The repository now includes reproducible CV diagnostics and report-generation tooling. The next steps are patient-level aggregation and final hold-out evaluation with calibrated thresholds.

References

1. Anomaly detection via reconstruction: https://arxiv.org/abs/2309.16053 (self-supervised / generative models for small sample size).

A Code extracts

Listing 1: Comparison script: both models on the same validation set (compare_patch_classifiers.py).

```
    # CNN
    cnn = SimpleCNN(num_classes=2).to(device)
    cnn.load_state_dict(torch.load(args.cnn_checkpoint, ...))
    cnn_probs, labels = compute_cnn_probs(cnn, val_loader, device)
    cnn_pred = (cnn_probs >= 0.5).astype(np.int64)

    # Autoencoder
    ae = PatchAutoencoder().to(device)
    ae.load_state_dict(torch.load(args.ae_checkpoint, ...))
    ae_errors, ae_labels = compute_reconstruction_errors(ae, val_loader, device, ...)
    ae_pred = (ae_errors > ae_threshold).astype(np.int64)

    acc_cnn, prec_cnn, cm_cnn = _metrics(labels, cnn_pred)
    acc_ae, prec_ae, cm_ae = _metrics(labels, ae_pred)
```

Listing 2: CNN for patch classification (model_classifier.py).

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes: int = 2) -> None:
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(64 * 32 * 32, 128), nn.ReLU(),
            nn.Linear(128, num_classes),
        )
    def forward(self, x): return self.classifier(self.features(x))
```

Listing 3: Autoencoder: Encoder, Decoder, and full model (model_autoencoder.py).

```
class Encoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1), nn.LeakyReLU(0.2), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.LeakyReLU(0.2), nn.MaxPool2d(2),
            nn.Conv2d(128, 256, 3, padding=1), nn.LeakyReLU(0.2), nn.MaxPool2d(2),
            nn.Conv2d(256, 512, 3, padding=1), nn.LeakyReLU(0.2), nn.MaxPool2d(2),
            nn.Conv2d(512, 512, 3, padding=1), nn.LeakyReLU(0.2), nn.MaxPool2d(2),
        )
    def forward(self, x): return self.conv(x)

class Decoder(nn.Module):
    def __init__(self):
        super().__init__()
```

```
        self.up = nn.Sequential(
            nn.ConvTranspose2d(512, 512, 2, stride=2), nn.LeakyReLU(0.2),
            nn.ConvTranspose2d(512, 256, 2, stride=2), nn.LeakyReLU(0.2),
            nn.ConvTranspose2d(256, 128, 2, stride=2), nn.LeakyReLU(0.2),
            nn.ConvTranspose2d(128, 64, 2, stride=2), nn.LeakyReLU(0.2),
            nn.ConvTranspose2d(64, 3, 2, stride=2), nn.Tanh(),
        )
    def forward(self, x): return self.up(x)

class PatchAutoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = Encoder()
        self.decoder = Decoder()
    def forward(self, x):
        return self.decoder(self.encoder(x))
```

Listing 4: AE per-sample reconstruction error (`model_autoencoder.py`). `mean`: pixel MSE; `max_local`: max over 8×8 cell means.

```
def reconstruction_error_per_sample(model, x, aggregation="mean"):
    x_hat = model(x)
    err = (x - x_hat).pow(2).mean(dim=1, keepdim=True) # (B, 1, H, W)
    if aggregation == "mean":
        return err.view(x.size(0), -1).mean(dim=1)
    if aggregation == "max_local":
        B, _, H, W = err.shape
        cell = 8
        nc_h, nc_w = H // cell, W // cell
        err = err.view(B, 1, nc_h, cell, nc_w, cell).mean(dim=(3, 5))
        return err.view(B, -1).max(dim=1)[0]
```

Listing 5: ROC-based patch threshold: closest to (FPR=0, TPR=1) (`roc_threshold_optimal`, `model_autoencoder.py`).

```
def roc_threshold_optimal(errors, labels, positive_class=1):
    order = np.argsort(-errors)
    sorted_errors, sorted_labels = errors[order], (labels[order] == positive_class).
        astype(np.float64)
    n_pos, n_neg = sorted_labels.sum(), len(sorted_labels) - n_pos
    tpr = np.cumsum(sorted_labels) / n_pos
    fpr = np.cumsum(1.0 - sorted_labels) / n_neg
    thresh = np.concatenate([[sorted_errors[0]+1], sorted_errors, [sorted_errors
        [-1]-1]])
    thresh = (thresh[:-1] + thresh[1:]) / 2
    dist = fpr**2 + (1 - tpr)**2
    idx = np.argmin(dist)
    return float(thresh[idx]), float(tpr[idx]), float(fpr[idx])
```

Listing 6: Patient-level aggregation.

```
for patient in patients:
    patches = load_all_cropped_patches(patient)
    patch_scores = [model_score(p) for p in patches] # CNN prob or AE error
    contaminated = [1 if s > tau_patch else 0 for s in patch_scores]
    ratio = sum(contaminated) / len(contaminated)
    patient_pred = 1 if ratio > tau_patient else 0
```

Listing 7: Slurm training and comparison commands.

```
sbatch train_classifier.sbatch.sh
sbatch train_autoencoder.sbatch.sh
python compare_patch_classifiers.py
sbatch patient_cnn_fold.sbatch.sh
sbatch patient_autoencoder_fold.sbatch.sh
sbatch patient_plots.sbatch.sh
```